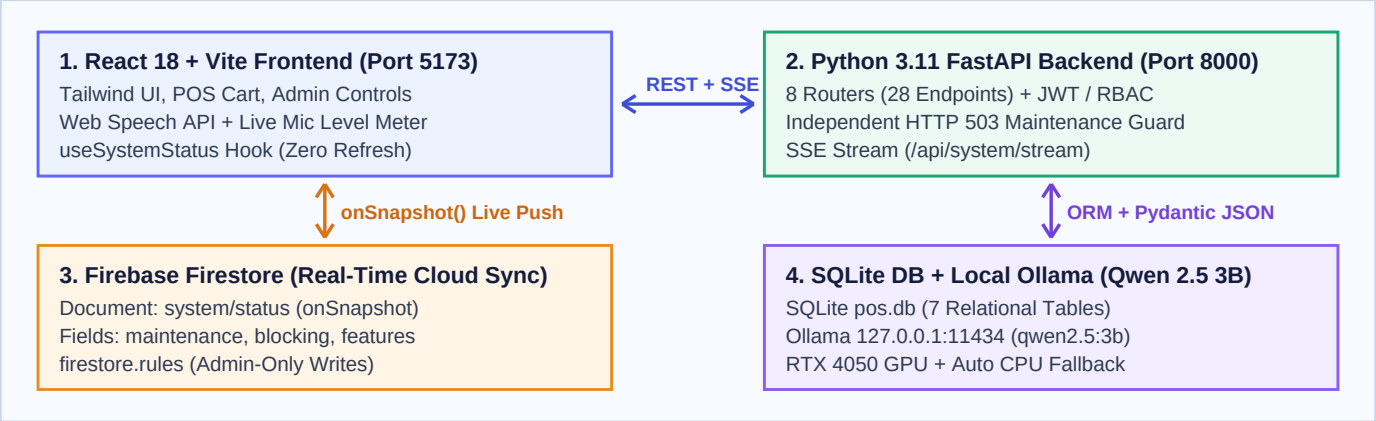
COMPLETE SYSTEM ARCHITECTURE, HOW IT WORKS & TECHNOLOGIES

Voice POS - Full System Guide & Antigravity Prompt

End-to-End Architecture: React 18 + FastAPI + SQLite + Local Ollama (Qwen 2.5 3B) + Firebase Firestore

1. Full System Architecture Overview

Voice POS is a privacy-first, 100% local AI Voice-Controlled Point of Sale (POS), Inventory Management, and Real-Time Operations web application built for small shops and SMEs in Sri Lanka.



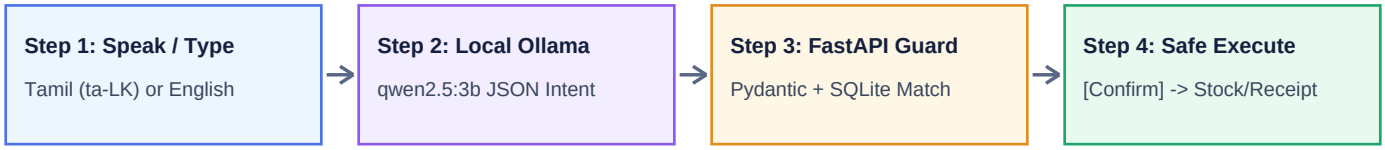
2. Complete Technology Stack Used Right Now

Layer	Technologies & Versions	What It Does in Your System
Frontend UI	React 18.3, Vite 5.4, JSX, Tailwind CSS 3.4, React Router v6, Lucide	Full-screen responsive English POS interface: Dashboard, POS Cart, Products, Inventory, Sales, Reports, Voice Console, Settings & Admin Controls.
Speech & Audio	Browser Web Speech API + Web Audio API	Continuous SpeechRecognition (en-US & ta-LK Tamil), live 0-100% microphone input meter, mic selector, and SpeechSynthesis TTS audio replies.
Backend API	Python 3.11, FastAPI >=0.115, Uvicorn >=0.32	28 REST & SSE endpoints across 8 routers (/api/auth, /config, /system, /products, /inventory, /sales, /voice, /reports) + /api/health.
Database & ORM	SQLite (pos.db), SQLAlchemy >=2.0.35, Pydantic v2 >=2.9	7 relational tables with check constraints, foreign keys, soft-deletes, immutable stock ledger, and strict Pydantic validation.
Local AI Engine	Ollama (127.0.0.1:11434), ollama SDK >=0.4.7, qwen2.5:3b	100% offline Tamil & English command understanding via structured JSON Schema. Zero external cloud LLM APIs or API keys.
Hardware Tuning	NVIDIA RTX 4050 (6GB VRAM), 16GB RAM, GPU/RAM telemetry	Quantized 1.9 GB model, compact num_ctx=2048, single warm model (keep_alive=10m), and automatic CPU fallback (num_gpu=0).
Real-Time Ops	Firebase Web SDK ^13.2 (Firestore onSnapshot) + FastAPI SSE	Monitors system/status via useSystemStatus hook; switches banner/full-screen lock & 5 feature flags live without F5 refresh.

Layer	Technologies & Versions	What It Does in Your System
Security & Auth	PyJWT >=2.9, bcrypt >=4.2, python-dotenv, firestore.rules	JWT Bearer auth, bcrypt password hashing, Owner/Admin vs Staff RBAC (HTTP 403), HTTP 503 API guard, and message sanitization.

3. How Every Workflow Works Step-by-Step

A. Local AI Voice Command & Two-Step Confirmation Pipeline



1. Voice / Text Capture: Shopkeeper speaks naturally in Tamil (ta-LK) or English (en-US) on the Voice Assistant page while Web Audio API displays live mic volume (0-100%), or types a command manually.
2. Local Qwen 2.5 3B Inference: FastAPI passes the active SQLite product catalog (including Sri Lankan Tamil aliases like Arisi, Seeni, Paal, Theneer) and transcript to local Ollama (qwen2.5:3b) enforcing AIStructuredIntent JSON output.
3. Pydantic Validation & SQLite Lookup: FastAPI validates the JSON with Pydantic and matches the product against SQLite. AI never runs SQL directly. Read-only intents (CHECK_STOCK, CHECK_PRICE, CHECK_LOW_STOCK, GET_TODAY_SALES, SEARCH_PRODUCT) answer immediately.
4. Mandatory [Confirm] / [Cancel] Execution: Stock/sale mutations (ADD_STOCK, REDUCE_STOCK, RECORD_SALE) save a PENDING_CONFIRMATION row with a unique token. Only when the user clicks [Confirm] does FastAPI re-verify live stock, update Product.stock_quantity, log StockTransaction, and generate a receipt.

B. Manual POS Checkout, BIG_AMOUNT Guard & Stock Ledger

1. Cart & Validation: Cashiers search items by name, category, or barcode in LKR (Rs.). Insufficient stock is blocked automatically, and bills exceeding BIG_AMOUNT (Rs. 50,000) show a confirmation warning.
2. Atomic Checkout & Receipt: POST /api/sales atomically writes Sale (INV-YYYYMMDD-XXXX) + SaleItem rows, deducts Product.stock_quantity, writes SALE rows to stock_transactions, calculates cash change, and opens a printable thermal receipt.

C. Zero-Refresh Real-Time Maintenance Mode & Feature Flags

1. Dual Push Channels (No Browser Refresh): useSystemStatus subscribes simultaneously to Firestore document system/status via onSnapshot() AND FastAPI's Server-Sent Events stream (GET /api/system/stream).
2. In-Memory Screen Swap & Independent HTTP 503 Guard: Enabling Banner Mode mounts MaintenanceBanner; Blocking Mode swaps non-admin tabs to MaintenanceScreen in memory and restores POS automatically when turned off. FastAPI's make_maintenance_guard independently returns HTTP 503 for protected APIs while keeping /api/auth/* and /api/system/* open for admins.

4. SQLite Database Schema (7 Tables) & API Routers

SQLite Table	Key Columns	Purpose
users	id, username, hashed_password, role	Owner/admin and staff accounts with bcrypt password hashes.
products	product_id, product_name, tamil_name, selling_price, stock_quantity, low_stock_limit	Shop catalog, LKR pricing, measurement units, and low-stock limits.
sales	id, invoice_number, user_id, total_amount, cash_received, change_amount, source	Completed POS and VOICE sales invoices in LKR.

SQLite Table	Key Columns	Purpose
sale_items	id, sale_id, product_id, quantity, unit_price, subtotal	Line items belonging to each Sale invoice.
stock_transactions	id, product_id, transaction_type, quantity, previous_stock, new_stock	Immutable audit ledger (INITIAL, ADD, REDUCE, SALE, ADJUSTMENT).
voice_commands	id, confirmation_token, transcript, intent, status, validated_payload	Voice AI history and pending confirmation token store.
system_status	id=1, maintenance, blocking, message, estimated_time, features_json	Authoritative maintenance state and 5 JSON feature flags.

Frontend Pages & Backend FastAPI Routers (28 Endpoints)

Frontend Route / Page	FastAPI Router	Key Responsibilities & Guards
/login (LoginPage.jsx)	/api/auth	POST /register, /login, /logout, GET /me. Open during maintenance for Admin login.
/ (DashboardPage.jsx)	/api/reports	GET /dashboard live KPIs, low-stock alerts, recent sales & quick product lookup.
/pos (PosPage.jsx)	/api/sales	POST /sales atomic checkout, BIG_AMOUNT alert, cash change & thermal receipt.
/products (ProductsPage.jsx)	/api/products	Full CRUD for shop products, categories, barcodes, units & low-stock limits.
/inventory (InventoryPage.jsx)	/api/inventory	GET /low-stock, POST /adjust stock additions/reductions, GET /transactions.
/voice (VoiceAssistantPage.jsx)	/api/voice	POST /process (local Qwen 2.5 3B), POST /confirm, POST /cancel, GET /history.
/settings (SettingsPage.jsx)	/api/config	GET /config public currency/language, local Ollama health & RTX GPU/RAM status.
/admin (AdminControlPage.jsx)	/api/system	GET /status, PUT /status (Admin RBAC), GET /stream (SSE) & HTTP 503 guards.

5. Copy-Ready Master Prompt for Google Antigravity

Copy and paste the prompt below directly into Antigravity to explain, extend, or recreate this system:

```
# ANTIGRAVITY PROJECT PROMPT - VOICE POS (LOCAL OLLAMA + FIREBASE REAL-TIME OPS)

Build or extend "Voice POS", a complete, production-ready AI Voice-Controlled Point of
  Sale (POS) and Inventory Management web application designed for small shops and SMEs
  in Sri Lanka.

## 1. EXACT TECHNOLOGY STACK CURRENTLY USED

### Backend (Python 3.11 + FastAPI)
- Python 3.11, FastAPI (>=0.115.0), Uvicorn (>=0.32.0)
- SQLite database (pos.db) + SQLAlchemy 2.0 ORM (>=2.0.35)
- Pydantic v2 (>=2.9.0) for strict request/response & AI JSON validation
- Authentication & Security: PyJWT (>=2.9.0) Bearer tokens + bcrypt (>=4.2.0) password
  hashing
- Environment Configuration: python-dotenv (>=1.0.1) validated at startup
- Testing & HTTP Client: pytest (>=8.3.0), httpx (>=0.27.0)

### Local AI Engine (Mandatory - 100% Local Inference)
- Runtime: Ollama running locally at http://127.0.0.1:11434
- SDK: Official ollama Python library (>=0.4.7)
- Model: Qwen 2.5 3B (qwen2.5:3b) - quantized ~1.9 GB local model
- Hardware Optimization: Tuned for NVIDIA RTX 4050 (6GB VRAM) + 16GB RAM (num_ctx=2048,
  num_predict=256, keep_alive="10m", live GPU/RAM telemetry, automatic CPU fallback
  num_gpu=0)
- Strict Constraint: NEVER use Gemini, OpenAI, Claude, or any external cloud LLM API. AI
  never executes SQL directly; it only outputs structured JSON intents validated against
  SQLite records.

### Frontend (React 18 + Vite + Tailwind CSS)
- React 18.3.1, Vite 5.4, JavaScript (JSX / ES Modules)
- Tailwind CSS 3.4 (PostCSS + Autoprefixer), Lucide React icons (0.460.0)
- React Router v6 (6.28.0) + centralized Fetch API client
- Browser Web Speech API (continuous SpeechRecognition for en-US & ta-LK +
  SpeechSynthesis TTS)
- Web Audio API live 0-100% microphone input level meter & input device selector

### Real-Time Maintenance Mode & Feature Flags
- Firebase Web SDK (^13.2.0): Firestore document "system/status" monitored in real time
  via onSnapshot() inside the reusable useSystemStatus React hook.
- Firestore Security Rules (firestore.rules): Public client read on system/status; writes
  restricted strictly to authenticated admins. Zero Firebase Admin/private keys in
  frontend.
- Backend Zero-Refresh Stream & Enforcement: FastAPI Server-Sent Events
  (/api/system/stream) + independent make_maintenance_guard returning HTTP 503 Service
  Unavailable (MAINTENANCE_MODE / FEATURE_DISABLED) while keeping /api/auth/* and
  /api/system/* accessible to administrators.

## 2. ENVIRONMENT VARIABLES

backend/.env:
AI_PROVIDER=ollama
OLLAMA_BASE_URL=http://127.0.0.1:11434
OLLAMA_MODEL=qwen2.5:3b
OLLAMA_TIMEOUT=60
CURRENCY=LKR
LANGUAGE=en-US
LOW_STOCK_DEFAULT=5
BIG_AMOUNT=50000

frontend/.env:
```

```
VITE_FIREBASE_API_KEY=your_firebase_web_api_key
VITE_FIREBASE_AUTH_DOMAIN=your-project-id.firebaseapp.com
VITE_FIREBASE_PROJECT_ID=your-project-id
VITE_FIREBASE_STORAGE_BUCKET=your-project-id.appspot.com
VITE_FIREBASE_MESSAGING_SENDER_ID=your_messaging_sender_id
VITE_FIREBASE_APP_ID=your_firebase_web_app_id

## 3. CORE APPLICATION MODULES & RULES
1. Full-Screen English UI: Non-overlapping collapsible desktop sidebar, top bar with
  Fullscreen toggle & TTS mute, mobile drawer/bottom tabs. Pages: Dashboard, POS /
  Billing, Products, Inventory, Sales History, Reports, Voice Assistant (full-width
  console without Quick Sample Voice Commands panel), Settings, and admin-only Admin
  Controls (/admin).
2. POS & Billing (LKR / Rs.): Product search, cart controls, stock validation, BIG_AMOUNT
  (> Rs. 50,000) warning, cash change calculation, stock deduction ledger, and printable
  thermal receipts.
3. Safe Voice AI Pipeline: Supports Tamil & English commands (CHECK_STOCK, ADD_STOCK,
  REDUCE_STOCK, RECORD_SALE, CHECK_PRICE, CHECK_LOW_STOCK, GET_TODAY_SALES,
  SEARCH_PRODUCT). Requires explicit [Confirm] / [Cancel] approval before mutating
  inventory or recording sales. Manual POS works 100% offline even if Ollama is stopped.
4. Real-Time Admin Control Panel (/admin): Toggle maintenance on/off, switch Banner-Only
  vs Full-Screen Blocking without page refresh, sanitize public messages/duration, and
  toggle 5 feature flags (posBilling, voiceAssistant, productManagement,
  inventoryAdjustments, reportsAnalytics).
```